Accelerating Polynomial System Solving via Factorization and Unsatisfiable-Core Learning

Ilay Menahem, Michael Shalyt, Ido Kaminer

August 30, 2026

Abstract

Polynomial systems are a fundamental language for describing algebraic and geometric problems across mathematics and its applications. Beyond finding individual solutions, one often seeks global properties of the solution space, such as its dimension and irreducible components, or structural properties and decompositions of the underlying polynomial ideal. For large polynomial systems with many variables and high degrees, determining these global properties can become intractable. Leading approaches include variants of Buchberger's algorithm with syzygy, decomposition techniques, resultants, and cylindrical algebraic decomposition. However, even the most modern approaches still often fail on practical problems or slight perturbations of standard benchmarks. Here we introduce a framework that divides a polynomial system into simpler branches. It assumes only that the branch solver describes satisfiable branches and certifies unsatisfiable (UNSAT) ones. If the solver also returns an UNSAT core, the framework uses it to prune later branches, following the same basic idea as conflict-driven clause learning (CDCL). In our experiments, we use Gröbner-basis solvers and apply the method to two systems arising from research in quantum field theory for which no standard method in our comparison succeeds. Together, these ideas offer a path toward the exact analysis of previously inaccessible polynomial systems.

1 Introduction

Systems of multivariate polynomial equations appear whenever a symbolic model contains nonlinear algebraic constraints. Applications include robotics, coding theory, optimization, statistics, and physics [25]. In many of these problems, an isolated

solution is not enough. One needs the dimension of the solution set, its irreducible components, or exact relations that hold on every solution. These are questions about the variety and its defining ideal, rather than about a single point [7, 15].

Elimination by resultants provided an early systematic way to attack polynomial systems, but the introduction of Gröbner bases gave a common algorithmic framework for elimination, ideal membership, dimension, and related tasks [7, 15]. Buchberger developed this framework in his 1965 thesis [6]. Later algorithms changed how the expensive reductions are organized. F4 replaces individual polynomial reductions with sparse linear algebra, while F5 uses signatures to avoid many reductions to zero [9, 10]. Primary-decomposition algorithms use Gröbner bases and factorization to separate components [12]. Numerical algebraic geometry takes a different route: homotopy continuation computes isolated roots numerically, while witness sets represent positive-dimensional components [23, 24].

Recent work has also explored machine-learning methods for polynomial systems. Peifer et al. trained a reinforcement-learning policy for pair selection in Buchberger's algorithm [20]. Kera et al. trained transformers to compute Gröbner bases directly for a restricted class of zero-dimensional radical ideals [14].

Despite this progress, the worst-case complexity of Gröbner-basis computation is doubly exponential, and large systems can remain inaccessible to monolithic methods [19]. Existing methods do not generally break down large systems into smaller subproblems which are much more manageable.

Here, we introduce a solver-agnostic framework that divides a polynomial system into simpler branches by interreducing and factoring its equations. The framework enumerates the minimal hitting sets of the resulting factor sets; each hitting set contains at least one factor from every equation and therefore defines one branch of the original solution set. An exact branch solver then either describes that branch, including any positive-dimensional families, or certifies that it is unsatisfiable.

To avoid repeated work, the framework learns certified UNSAT cores and prunes later hitting sets that contain them, adapting ideas from polynomial UNSAT-core extraction and conflict-driven learning [26, 18, 17].

If a factor branch remains too expensive for the chosen solver, the framework can refine it by splitting a selected variable into the cases $x = 0$ and $x \neq 0$. This heuristic split can expose additional algebraic structure for the branch solver without discarding either case.

We prove that refinement and variable zeroing terminate and preserve the full solution set. The returned description is complete whenever every branch is resolved by the branch solver. We compare the resulting implementation with direct Gröbner-basis computation, symbolic decomposition, and numerical homotopy on generated

benchmarks. We also apply the framework to a system from quantum field theory: with the `facstd` backend, the framework completes the computation, whereas the other tested backends leave unresolved branches.

2 Preliminaries and notation

2.1 Polynomial systems, ideals, and varieties

Let K be a field, let $\overline{K}$ be its algebraic closure, and write $R = K[x_1, \ldots, x_n]$. The input is a finite system $P = \{p_1, \ldots, p_m\} \subseteq R$ of nonzero polynomials. Unless stated otherwise, a solution is a point in $\overline{K}^n$. For a finite set $Q \subseteq R$, its solution set is

$$V(Q) = \{a \in \overline{K}^n : q(a) = 0 \text{ for every } q \in Q\}.$$

We call Q satisfiable if $V(Q) \neq \emptyset$ and unsatisfiable (UNSAT) if $V(Q) = \emptyset$.

The ideal generated by Q, denoted $\langle Q \rangle$, consists of all sums $\sum_{q \in Q} h_q q$ with $h_q \in R$.

For $X \subseteq \overline{K}^n$, its vanishing ideal $I(X)$ contains every polynomial in R that vanishes on all of X. The radical of an ideal J is $\sqrt{J} = \{g \in R : g^r \in J \text{ for some } r \geq 1\}$. Hilbert's Nullstellensatz gives

$$I(V(Q)) = \sqrt{\langle Q \rangle}.$$

Thus $V(Q) = \emptyset$ exactly when $\langle Q \rangle = \langle 1 \rangle$. These facts connect solution sets with computations in polynomial ideals [7, 15].

2.2 Monomial orders and Gröbner bases

Let $\mathbb{M}_n = \{x^\alpha : \alpha \in \mathbb{N}^n\}$ be the monomials of R. A monomial order $\prec$ is a total order on $\mathbb{M}_n$ such that every nonempty subset has a least element and

$$u \prec v \quad \Longrightarrow \quad uw \prec vw \qquad (u, v, w \in \mathbb{M}_n).$$

For a nonzero polynomial p, the leading monomial $\mathrm{LM}(p)$ is the greatest monomial of p under $\prec$, $\mathrm{LC}(p)$ is its coefficient, and $\mathrm{LT}(p)$ is its leading term.

For an ideal J, define its leading-monomial ideal by

$$\mathrm{LM}(J) = \langle \mathrm{LM}(f) : f \in J \setminus \{0\} \rangle.$$

A finite set $G \subseteq J$ is a Gröbner basis of J if

$$\mathrm{LM}(J) = \langle \mathrm{LM}(g) : g \in G \rangle.$$

A finite set Q is *interreduced* if every element is monic and no term of one element is divisible by the leading monomial of another. Algorithm 1 interreduces a finite set of nonzero polynomials.

Algorithm 1 Interreduction

Require: A finite set $Q \subseteq R \setminus \{0\}$ and a monomial order $\prec$
Ensure: An interreduced set that generates $\langle Q \rangle$

while for $p, q \in Q$ there is a term cu of p such that $\mathrm{LM}(q) \mid u$ **do**
  $r \leftarrow p - \dfrac{cu}{\mathrm{LT}(q)} q$
  $Q \leftarrow Q \setminus \{p\}$
  if $r \neq 0$ **then**
    $Q \leftarrow Q \cup \{r\}$
  end if
end while
$Q \leftarrow \{p/\mathrm{LC}(p) : p \in Q\}$
return Q

Each loop iteration cancels one monomial u and introduces only monomials strictly smaller than u. The multiset extension of the monomial order is well-founded, so the loop terminates. interreduction preserves $\langle Q \rangle$ and $V(Q)$.

Every ideal has a unique reduced Gröbner basis for a fixed monomial order. The reduced basis is $\{1\}$ if and only if $J = \langle 1 \rangle$. Therefore, a Gröbner basis computation can describe a branch and detect when it is UNSAT [7, 15].

An UNSAT core of a system Q is a subset $U \subseteq Q$ with $V(U) = \emptyset$. An UNSAT core is minimal if there is no $U' \subsetneq U$ such that $V(U') = \emptyset$. Any system containing U is also UNSAT, since adding equations cannot introduce solutions.

2.3 Factor sets and hitting sets

Fix a factorization of each input polynomial over K:

$$p_i = a_i \prod_{f \in C_i} f^{e_{i,f}},$$

where $a_i \in K \setminus \{0\}$, C_i contains the distinct monic, nonconstant irreducible factors of p_i, and $e_{i,f} \geq 1$. Multiplicity does not affect the zero set. A point satisfies $p_i = 0$ exactly when at least one factor in C_i vanishes at that point.

A hitting set for $C_1, \ldots, C_m$ is a set $H \subseteq \bigcup_i C_i$ such that $H \cap C_i \neq \emptyset$ for every i. It is minimal if no proper subset is also a hitting set. We write $\mathcal{H}(P)$ for the family of minimal hitting sets obtained from P. Each $H \in \mathcal{H}(P)$ defines a branch in which all factors in H vanish [1].

2.4 Encoding nonzero variables

Let $N \subseteq \{x_1, \ldots, x_n\}$ be the variables required to be nonzero. Here are two ways to encode these constraints as polynomial equations.

One encoding introduces a separate auxiliary variable s_x for each $x \in N$ and works in the extended ring $R_{\text{sep}} = R[s_x : x \in N]$. For each $x \in N$, define the polynomial

$$\sigma_x = 1 - s_x x, \tag{1}$$

and impose the equation $\sigma_x = 0$.

The second encoding uses one auxiliary variable s, works in $R_{\text{prod}} = R[s]$, and combines all nonzero constraints into the polynomial

$$\sigma_N = 1 - s \prod_{x \in N} x, \tag{2}$$

with the equation $\sigma_N = 0$.

Let $Q \subseteq R$ and let π denote the appropriate coordinate projection onto the original $x_1, \ldots, x_n$ coordinates. In the first encoding,

$$\pi\left(V\left(Q \cup \{\sigma_x : x \in N\}\right)\right) = \{a \in V(Q) : a_x \neq 0 \text{ for every } x \in N\}.$$

Indeed, $\sigma_x = 0$ has a solution s_x exactly when $a_x \neq 0$. Similarly, the product encoding satisfies

$$\pi\left(V\left(Q \cup \{\sigma_N\}\right)\right) = \{a \in V(Q) : a_x \neq 0 \text{ for every } x \in N\}.$$

3 Branching framework

3.1 Framework description

Solving a system of polynomial equations is double exponential hard [19]. Thus breaking a system of polynomial equations into smaller subproblems allows for more efficient solving.

We break a system into smaller subproblems by iteratively doing two basic steps for each subproblem:

1. Interreduce the current subproblems.
2. Factor each generator and enumerate the minimal hitting sets of the factor sets.

Here is the pseudocode for the refinement of a polynomial system Q into a smaller set of subproblems.

Algorithm 2 Refinement

Require: A set of polynomials Q and A

Interreduce Q ▷ Optional
Factor each generator of Q
$\mathcal{H} \leftarrow$ the minimal hitting sets of the factor sets
$T \leftarrow \emptyset$
for all $H \in \mathcal{H}$ **do**
    if $H = Q$ or $H \in A$ **then**
        $T \leftarrow T \cup \{H\}$
    else
        $T \leftarrow T \cup \textsc{Refine}(H, A \cup \{\widetilde{Q}\})$
    end if
end for
return T

Let $\mathcal{L}(P)$ denote the terminal hitting sets obtained from P. For each $H \in \mathcal{L}(P)$, the branch solver must either describe $V(H)$ exactly or certify that $V(H) = \emptyset$.

The solver may also return a certified UNSAT core $U \subseteq H$, any later hitting set containing U can be determined to be UNSAT without calling the solver.

Algorithm 3 Branching

Require: A set of polynomials $P = \{p_1, \ldots, p_m\}$ and an exact branch solver $\mathcal{B}$.

$C \leftarrow \emptyset$ ▷ Certified UNSAT cores
$S \leftarrow \emptyset$ ▷ Solution-set representations
$R \leftarrow \textsc{Refine}(P, \emptyset)$
for all $H \in R$ **do**
    if H contains an UNSAT core from C **then**
        continue
    end if
    Use $\mathcal{B}$ to solve H
    if $\mathcal{B}$ returns a representation of $V(H)$ **then**
        Add the returned representation to S
    else
        if $\mathcal{B}$ returns a certified UNSAT core **then**
            Add the core to C
        end if
    end if
end for
return S, C

The order of the terminal hitting sets affects running time but not correctness. We use the number of generators in a hitting set as its estimated cost and process smaller sets first. An UNSAT core found in a small set may then prune larger sets before they reach the solver.

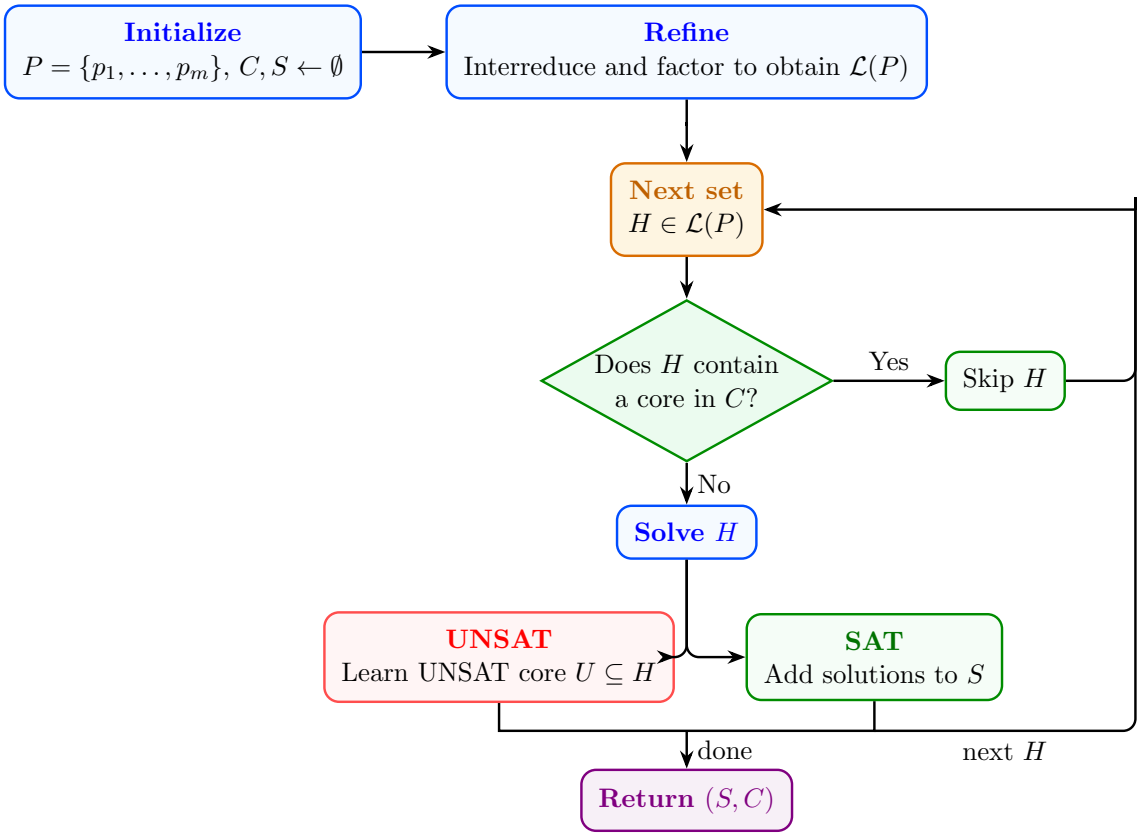


Figure 1: Flowchart of the branching procedure of Algorithm 3.

3.2 Theoretical analysis

Let S be a collection of solution-set representations, and $\mathrm{Sol}(S)$ be the union of the represented sets. The theoretical analysis does not require K to be algebraically closed. If a system has no solution over the algebraic closure, it has no solution over K.

Lemma 1 (Sufficiency of minimal hitting sets) *Let T be the sets of minimal hitting sets then*

$$\bigcup_{H \in T} V(H) = \bigcup_{\text{minimal hitting set } H \subseteq T} V(H).$$

Proof 1 *Let there be $S \in T$, repeatedly removing redundant factors from S eventually produces a minimal hitting set H. Since $H \subseteq S$ we have $V(S) \subseteq V(H)$ and*

$\bigcup_{H\in T} V(H) \subseteq \bigcup_{minimal\ hitting\ set H\subseteq T} V(H)$. *The other direction is trivial, so the two unions are equal.*

For a system Q, write $\mathcal{H}(Q)$ for the minimal hitting sets obtained in one factorization round.

Theorem 1 (One-round branch decomposition)

$$V(Q) = \bigcup_{H\in\mathcal{H}(Q)} V(H).$$

Proof 2 *Write $Q = \{q_1, \ldots, q_m\}$ and let F_i be the nonconstant irreducible factors of q_i. Take $x \in V(Q)$. Since $q_i(x) = 0$, at least one factor in F_i vanishes at x for every i. The set of all factors that vanish at x is a hitting set of $F_1, \ldots, F_m$. Because the factor sets are finite, this set contains a minimal hitting set H. Every factor in H vanishes at x, so $x \in V(H)$.*

For the reverse inclusion, take $x \in V(H)$ for some $H \in \mathcal{H}(Q)$. The set H intersects every F_i, so each q_i has a factor that vanishes at x. Hence $q_i(x) = 0$ for every i, and $x \in V(Q)$.

Lemma 2 (Recursive refinement) *Suppose refinement of Q terminates with the finite collection $\mathcal{L}(Q)$. Then*

$$V(Q) = \bigcup_{H\in\mathcal{L}(Q)} V(H).$$

Proof 3 *Algorithm 2 is a fixed-point recursion. Let us show that each recursive call preserves the zero set. The base case is no step of refinement, in which case the zero set is $V(Q)$. Let there be $L_n(Q)$, the collection of terminal hitting sets after n refinement steps.*

$$\begin{aligned} V(Q) &= \bigcup_{H\in\mathcal{L}_n(Q)} V(H) \\ &= \bigcup_{H\in\mathcal{L}_n(Q)} V(interreduction(H)) \\ &= \bigcup_{H\in\mathcal{L}_n(Q)} \bigcup_{H'\in\mathcal{H}(interreduction(H))} V(H') \\ &= \bigcup_{H\in\mathcal{L}_{n+1}(Q)} V(H). \end{aligned}$$

Theorem 2 (Termination of refinement) *Algorithm 2 terminates on every finite polynomial system.*

Proof 4 *If interreduction is disabled, every child consists of irreducible polynomials. Factoring it again returns the same system, so the condition $H = Q$ makes it terminal.*

Suppose interreduction is enabled. For an interreduced system Q, let

$$M(Q) = \langle \mathrm{LM}(q) : q \in Q \rangle.$$

For every hitting-set child H and every $q \in Q$, some $h \in H$ divides q. Hence $M(Q) \subseteq M(H)$. Equality is possible only when every selected factor is associate to the generator it hits; after normalization this gives $H = Q$, which is terminal. Interreduction can only enlarge $M(H)$, so M strictly increases along every recursive path.

The ascending-chain condition for monomial ideals rules out an infinite path. Every node has finitely many children, so König's lemma implies that the refinement tree is finite. Therefore Algorithm 2 terminates.

Here is a known result that justifies branch pruning using UNSAT cores.

Lemma 3 *Let U be an UNSAT core. If $U \subseteq H$, then $V(H) = \emptyset$.*

The following theorem allows the use of Algorithm 3 without loss of correctness.

Theorem 3 *Algorithm 3 is sound and complete for P.*

Proof 5 *By Theorem 2, refinement terminates with a finite collection $\mathcal{L}(P)$. Every interreduction round preserves the current ideal, and every factorization round preserves the zero set by Theorem 1. Lemma 2 therefore gives*

$$V(P) = \bigcup_{H \in \mathcal{L}(P)} V(H).$$

Now we shall show that the algorithm terminates.

Theorem 4 *If the exact solver is known to terminate then Algorithm 3 terminates.*

Proof 6 *By Theorem 2, the call $R = \textsc{Refine}(P, \emptyset)$ terminates and returns a finite collection. Therefore Algorithm 3 calls the exact solver a finite number of times, and each call terminates by assumption. Thus Algorithm 3 terminates.*

4 Branching with variable zeroing

4.1 Branching with variable zeroing

Lets say that the solver doesn't terminate within reasonable amount of time for an hitting set given by Algorithm 3.

In this case, we can further simplify the system by branching further. In each branch assume some side condition holds, and letting Algorithm 3 run on the simplified system. This splitting can be done recursively until the solver terminates or the computation budget is exhausted.

For completeness, the union of the side conditions must cover all possibilities. A set of conditions is a good choice if it is helpful for your solver, and works well with algorithm 2.

For this work the branching conditions is whether a variable is zero or nonzero. This simplification choice is used in other works [**?**]. in this work we considered physical systems and determined through past experience that we expect a variable to be always zero, or always nonzero.

Algorithm 4 Recursive branching with variable zeroing

Require: A polynomial system P and a timeout τ

function ZeroingBranching(P, τ)
  $X \leftarrow$ Variables(P)
  return SolveByZeroing($P, X, \emptyset, \emptyset, \tau, \emptyset$)
end function
function SolveByZeroing(Q, X, Z, N, τ, C)
  $(S, T) \leftarrow$ BranchingWithTimeout(Q, τ, C)
  $U \leftarrow \emptyset$ ▷ Unresolved branches
  for all $R \in T$ **do**
    if $X = \emptyset$ **then**
      $U \leftarrow U \cup \{(R, Z, N)\}$
    else
      $x \leftarrow$ ChooseVariable(R, X)
      $R_0 \leftarrow$ Substitute($R, x, 0$)
      $R_1 \leftarrow$ UpdateSaturation($R, N \cup \{x\}$)
      $(S_0, U_0, C) \leftarrow$ SolveByZeroing($R_0, X \setminus \{x\}, Z \cup \{x\}, N, \tau, C$)
      $(S_1, U_1, C) \leftarrow$ SolveByZeroing($R_1, X \setminus \{x\}, Z, N \cup \{x\}, \tau, C$)
      $S \leftarrow S \cup S_0 \cup S_1$
      $U \leftarrow U \cup U_0 \cup U_1$
    end if
  end for
  return (S, U, C)
end function

For Algorithm 4 we use a global set C of learned UNSAT cores.

Let N be the variables declared nonzero on the current path. Subsection 2.4 describes two ways to encode these conditions as polynomial equations.

Solutions of the nonzero branch are projected onto the original variables, so the saturation variable does not appear in the output. The zero and nonzero decisions remain part of the solution-set representation.

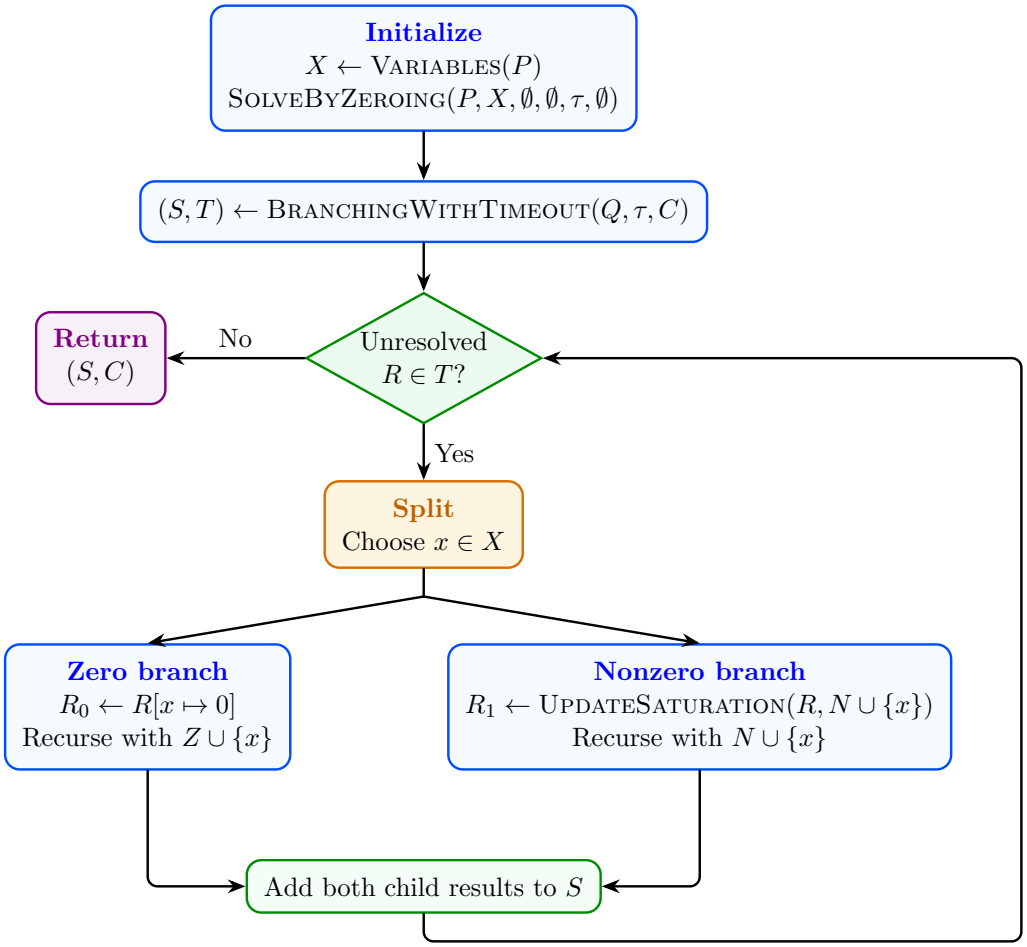


Figure 2: Flowchart of the recursive variable-zeroing procedure of Algorithm 4.

Running the method on `systems/system.txt` with M4GB and a five-second timeout per hitting set produces the recursion tree in Figure 3.

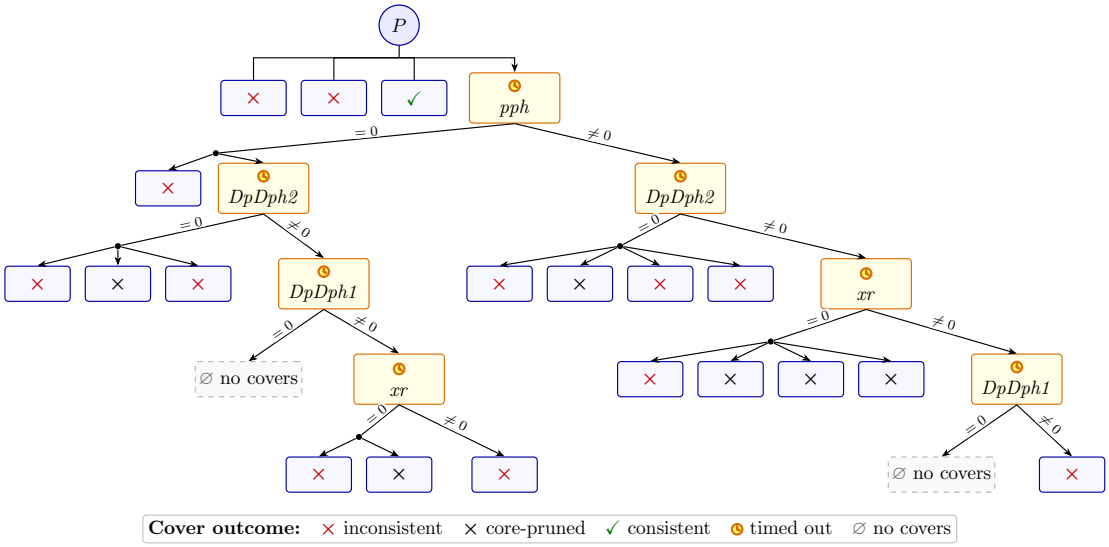


Figure 3: Recursion tree for `systems/system.txt`.

4.2 Choice of variable for splitting

The choice of variable for splitting affects the efficiency of the algorithm. Every split creates at least two new hitting sets to check. If a variable barely occurs in the current hitting set, both recursive calls may repeat most of the same work. A variable that occurs in many generators is a more promising choice. Setting it to zero removes all terms containing it at once. In the nonzero branch, it is appended to the product in Equation (2).

For each candidate variable x that is still available on the current branch, define

$$\begin{aligned} g_Q(x) &= \#\{q \in Q : x \text{ occurs in } q\}, \\ t_Q(x) &= \#\{(q, m) : q \in Q,\ m \text{ is a term of } q,\ \deg_x(m) > 0\}, \\ e_Q(x) &= \sum_{q \in Q} \sum_{\substack{m \text{ a term of } q, \\ \deg_x(m) > 0}} \deg_x(m). \end{aligned}$$

Its score is the triple

$$h_Q(x) = \big(g_Q(x), t_Q(x), e_Q(x)\big),$$

and scores are compared lexicographically.

4.3 Branch contraction

Let $\mathcal{H}_0$ and $\mathcal{H}_1$ be the minimal hitting sets for $x = 0$ and $x \neq 0$. Suppose, up to exchanging the two children, that

$$\mathcal{H}_0 = \emptyset \qquad \text{and} \qquad \mathcal{H}_1 = \{H\}.$$

The first child is inconsistent, and we have

$$V_Y(Q) = V_Y(H).$$

There is no need to run the branch solver on H before choosing the next splitting variable.

4.4 Theoretical analysis of variable zeroing

We prove the correctness and termination of Algorithm 4.

Let $V_Y(Q; Z, N)$ denote the projection onto Y of the solutions of Q satisfying $z = 0$ for $z \in Z$ and $y \neq 0$ for $y \in N$. We also denote $Q_0 = Q|_{x=0}$ and $Q_1 = \textsc{UpdateSaturation}(Q, N \cup \{x\})$.

Theorem 5 (Correctness of a zeroing split) *Let there be $Z, N \subseteq Y$ such that $Z \cap N = \emptyset$, and let $x \in Y \setminus (Z \cup N)$. Then*

$$V_Y(Q; Z, N) = V_Y(Q_0; Z \cup \{x\}, N) \cup V_Y(Q_1; Z, N \cup \{x\}).$$

Proof 7 *Take a point in $V_Y(Q; Z, N)$. Its x-coordinate is either zero or nonzero. In the first case, substituting $x = 0$ gives a solution of Q_0 satisfying $Z \cup \{x\}$ and N. In the second case, the point satisfies the nonzero conditions $N \cup \{x\}$, which are encoded by Q_1. Thus it belongs to the corresponding set on the right.*

Conversely, a point in either set on the right satisfies Q, together with the conditions in Z and N. Therefore it belongs to $V_Y(Q; Z, N)$.

Theorem 6 (Correctness of variable zeroing) *If Algorithm 4 returns (S, U, C) on input P, then*

$$V(P) = \mathrm{Sol}(S) \cup \bigcup_{(R, Z_R, N_R) \in U} V_Y(R; Z_R, N_R).$$

Proof 8 *We prove the claim for every call to* SolveByZeroing*(b)y induction on the remaining recursion tree. For a call with input* (Q, X, Z, N, τ, C)*,* BranchingWithTimeout*(p)artitions the constrained zero set into the solutions it finds and the unresolved hitting sets:*

$$V_Y(Q; Z, N) = \mathrm{Sol}(S) \cup \bigcup_{R \in T} V_Y(R; Z, N).$$

This follows from Theorem 3; a timeout leaves the corresponding hitting set in T*, and pruning by a certified UNSAT core removes only an empty zero set.*

If $X = \emptyset$*, the algorithm adds each* $R \in T$ *to* U *with the current conditions, so the required equality follows. Otherwise, for every* $R \in T$ *the algorithm chooses* $x \in X$*. By Theorem 5,*

$$V_Y(R; Z, N) = V_Y(R_0; Z \cup \{x\}, N) \cup V_Y(R_1; Z, N \cup \{x\}).$$

The induction hypothesis applies to both recursive calls. Their solved and unresolved sets therefore replace $V_Y(R; Z, N)$ *without changing the union. Taking the union over all* $R \in T$ *proves the claim for the current call.*

The initial call has $Q = P$ *and* $Z = N = \emptyset$*, so its constrained zero set is* $V(P)$*. This gives the stated equality.*

The following corollary is a direct consequence of Theorem 6. It states that if all branches are resolved, Algorithm 4 returns a complete solution set.

Corollary 1 (Completeness of variable zeroing) *If* $U = \emptyset$ *in the output of Algorithm 4, then*

$$\mathrm{Sol}(S) = V(P).$$

Lastly we prove that Algorithm 4 terminates.

Theorem 7 (Termination of variable zeroing) *Algorithm 4 terminates.*

Proof 9 *We prove that* SolveByZeroing*(t)erminates by induction on* $|X|$*. First, every call to* BranchingWithTimeout*(t)erminates. Indeed, Theorem 2 shows that Algorithm 2 terminates, and returns finitely many hitting sets. On there minimal hitting sets, the solver either returns or is stopped after the timeout* τ*. Thus the call returns finite sets* S *and* T*.*

If $X = \emptyset$*, the algorithm loops over the finite set* T *and returns. Suppose now that* $|X| > 0$*. For each* $R \in T$*, the algorithm makes two recursive calls, both with* $X \setminus \{x\}$*. These calls terminate by the induction hypothesis. Since* T *is finite, the current call also terminates.*

The initial set Variables(P) *is finite, so the initial call to* SolveByZeroing*(t)erminates. Therefore Algorithm 4 terminates.*

5 Experiments

5.1 Setup

We implemented Algorithm 3 in Rust 1.97.1 and used Symbolica 1.5 for polynomial arithmetic over the rationals [22]. In all experiments, we solve the branches with a Gröbner-basis computation. The available implementations are M4GB, F5, GVW, F4, and Singular 4.4.1 [16, 10, 11, 9, 8]. A basis containing 1 certifies that the branch is UNSAT. We obtain an UNSAT core by iterative refinement as in Section 5 of [26]. The comparison also includes six direct whole-ideal Gröbner-basis configurations, three decomposition methods from Singular, Macaulay2 1.26.06, and a numerical method from HomotopyContinuation.jl under Julia 1.12.6 [8, 13, 5, 2].

For Algorithm 4, we test M4GB, F5, GVW, Symbolica, and Singular as branch solvers, disable UNSAT-core learning, and impose a three-second limit on each cover computation. We selected this cutoff by trying one, two, three, and five seconds on the hard gated systems. Three seconds is the smallest tested value that lets Singular complete the Katsura-8 residual (about 2.6 seconds); five seconds completes no additional hard system and only delays the subsequent splits.

We ran the benchmark on an Apple M4 Pro with 14 CPU cores and 24 GB of memory. Every table entry is a single run. Every method received 60 seconds per system; the three-second zeroing limit is an inner, per-cover limit within that budget.

For branching and variable zeroing, T is end-to-end wall time and F is the time from the start of the minimal-hitting-set search to its first satisfiable hitting set. Factorization and input processing are therefore included in T but not in F. The complete measurements are stored in the appendix.

5.2 Baseline systems

The baseline has 20 generated systems in six families.

- Signed-distance cycles and affine-factor hypergraphs test many inexpensive hitting sets.
- The overlapping random multiples use seeded dense quadratic factors that recur across equations, with no factor shared by every equation.
- For gated systems, we create a new variable and multiply every equation in Cyclic, Katsura, or dense quadratic by it [3, 4].
- This produces one easy selector cover and one nonlinear residual cover.

Family	Tested sizes	Variables/equations; hitting sets
Signed-distance cycle	$n = 8, 10, 12$	$8/8, 10/10, 12/12$; $8, 8, 32$
Overlapping random multiples	$(n, p, k) = (4, 6, 2), (6, 8, 3), (8, 10, 3)$	n/n; $4, 16, 28$
Positive affine hypergraph	$(n, m, k) = (8, 6, 4), (10, 8, 5), (12, 10, 6)$	n/m; $63, 239, 833$
Gated Cyclic	orders $5, 6, 7$	$6/5, 7/6, 8/7$; 2 each
Gated Katsura and dense quadratics	Katsura $6, 7, 8$; dense $n = 5, 6, 7$	$8/7, 9/8, 10/9, 6/5, 7/6, 8/7$; 2 each
Square affine hypergraph	$(n, k) = (8, 4), (10, 5)$	n/n; $31, 304$

Table 1: The generated systems used for the baseline benchmark.

The overlapping-random-multiple runs disable interreduction so that the generated factor-incidence pattern reaches the hitting-set enumerator unchanged. All other branching runs use interreduction. Table 2 selects the fastest completed branching backend, the fastest completed zeroing backend observed, and the fastest completed method in each baseline class. A branching or zeroing entry gives T/F in seconds; the other entries give total wall time. "Timeout" means that every tested method in that class timed out. "Partial" means that zeroing returned at least one unresolved branch, and "error" records a failed run rather than a mathematical conclusion.

System	Best branching T/F	Selected zeroing T/F	Best direct GB	Best decomposition	Numerical
Signed distance, $n = 8$	Symb. 0.008/0.000	Symb. 0.008/0.001	Symb. 0.008	facstd 0.190	32.410
Signed distance, $n = 10$	Symb. 0.025/0.002	Symb. 0.025/0.002	Symb. 0.029	facstd 0.430	29.944
Signed distance, $n = 12$	Symb. 0.087/0.002	Symb. 0.091/0.003	Symb. 0.472	primdec 3.056	36.880
Overlapping $(4, 6, 2)$	F5 0.008/0.000	F5 0.009/0.001	Symb. 0.413	facstd 0.265	30.069
Overlapping $(6, 8, 3)$	F5 0.132/0.001	F5 0.132/0.021	timeout	timeout	timeout
Overlapping $(8, 10, 3)$	Singular 2.931/0.010	Singular 2.892/0.152	timeout	timeout	timeout
Positive affine $(8, 6, 4)$	F5 0.008/0.000	Singular 0.181/0.012	timeout	facstd 0.368	50.934
Positive affine $(10, 8, 5)$	F5 0.020/0.000	Symb. 0.060/0.007	timeout	facstd 1.836	timeout
Positive affine $(12, 10, 6)$	F5 0.067/0.000	Singular 9.600/0.078	timeout	facstd 53.318	timeout
Gated Cyclic-5	Symb. 0.009/0.000	Symb. 0.011/0.001	Symb. 0.008	facstd 0.181	35.054
Gated Cyclic-6	Singular 0.045/0.008	Singular 0.043/0.009	Singular 0.309	M2 2.620	37.314
Gated Cyclic-7	timeout	Singular partial 24.118/0.009	timeout	timeout	58.287
Gated Katsura-6	Singular 0.049/0.008	Singular 0.049/0.009	GVW 0.226	facstd 0.233	31.532
Gated Katsura-7	Singular 0.283/0.008	Singular 0.284/0.009	GVW 3.176	facstd 0.539	31.980
Gated Katsura-8	Singular 2.657/0.052	Singular 2.588/0.009	timeout	facstd 3.596	34.264
Gated dense, $n = 5$	Singular 0.029/0.008	Singular 0.029/0.008	M4GB 0.171	facstd 0.198	29.289
Gated dense, $n = 6$	Singular 0.546/0.008	Singular 0.535/0.008	M4GB 4.736	facstd 0.755	30.053
Gated dense, $n = 7$	Singular 42.020/0.017	error	timeout	facstd 48.820	32.261
Square affine $(8, 4)$	Singular 0.470/0.023	Singular 0.445/0.012	timeout	facstd 0.601	timeout
Square affine $(10, 5)$	Symb. 1.174/0.000	Symb. 1.326/0.015	timeout	facstd 2.141	timeout
#systems solved	19	19	10	17	14
#fastest (ties counted)	12	10	2	0	2

Table 2: Best completed baseline results and the fastest completed zeroing backend observed. All times are in seconds.

Branching is a good choice when factorization produces branch ideals that are substantially easier than the expanded input ideal. A large number of hitting sets can still be manageable when branch solving and core checks are cheap; the positive affine family shows that the hitting-set count alone is not decisive. The method is

less effective when one cover retains the difficulty of the original nonlinear system, as Cyclic-7 does.

The first-solution measurements support this explanation. On positive affine $(12, 10, 6)$, F rounds to 0.000 seconds despite the 833 hitting sets. On gated Katsura-8, the first satisfiable cover appears in 0.052 seconds, but the full run takes 2.657 seconds. Factorization therefore finds the easy component quickly, while the nonlinear residual controls completion time.

Variable zeroing is intended for a narrower case: a cover has timed out, but branching a variable to zero or nonzero, both simplify it the system. The current benchmark gives limited evidence for that mechanism because none of the completed zeroing runs requires a split. On Cyclic-7, zeroing continues the stalled search and returns a partial result, but it does not complete the residual branch.

5.3 Physical Systems

The system is an order five approximation of the theories that represent a graviton are consistent with experimental results of QFT (quantum field theory) [21, 27]. Solving this system can lead to a better understanding of how quantum gravity may look like [27].

For this larger system, zeroing uses a one-second limit per cover and a 60-second total runtime limit. Ordinary branching uses a 60-second limit per cover and a 65-second external hard stop. Thus, an earlier completed cover can make the reported branching wall time slightly exceed 60 seconds.

Backend	Branching T/F (s)	Branching result	Zeroing T/F (s)	Zeroing result
M4GB	60.793/0.003	partial: 32 SAT, 1 timeout; $d = 14$	*timeout*/1.052	partial: 7 comp., 52 splits, 11 unresolved
F5	61.422/0.003	partial: 32 SAT, 1 timeout; $d = 14$	*timeout*/2.059	partial: 13 comp., 63 splits, 9 unresolved
GVW	61.653/0.003	partial: 32 SAT, 1 timeout; $d = 14$	*timeout*/2.071	partial: 13 comp., 60 splits, 9 unresolved
Symbolica	60.218/0.003	partial: 32 SAT, 1 timeout; $d = 14$	*timeout*/1.051	partial: 7 comp., 56 splits, 12 unresolved
Singular	60.342/0.039	partial: 32 SAT, 1 timeout; $d = 14$	*timeout*/0.566	partial: 1 comp., 44 splits, 12 unresolved
`facstd`	1.067/0.013	complete: 32 SAT, 1 UNSAT; $d = 14$	0.824/0.077	complete: 6 comp., 0 splits, 0 unresolved

Table 3: Physical-system results from the fresh single-run benchmark.

The `facstd` backend is the only tested backend that completes its run on the physical system, and it does so with both ordinary branching and variable zeroing. Every other backend finds the full solution set, but can't certify that the remaining branches are UNSAT.

6 Conclusion

We introduced a solver-agnostic framework that decomposes polynomial systems into minimal hitting sets of their factors. It prunes branches with certified UNSAT cores and splits timed-out branches into the cases $x = 0$ and $x \neq 0$.

We proved that refinement and variable zeroing terminate and preserve the full solution set. The output is complete whenever all branches are resolved.

The method combines a novel yet simple method for breaking down polynomial systems into smaller subproblems with factorization and interreduce. We add reusable UNSAT cores, and zero/nonzero splitting without depending on a particular branch solver.

We shown the method's effectiveness on benchmark systems and the larger the system, the larger the speedup. additionally, we showed that the method is the only one that can solve a QFT system in a reasonable time frame.

Future work will consider better case splitting strategies besides variable zeroing, and better ways to keep track and generate UNSAT cores.

References

[1] Claude Berge. *Hypergraphs: Combinatorics of Finite Sets*. North-Holland, Amsterdam, 1989.

[2] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017.

[3] Göran Björck and Ralf Fröberg. A faster way to count the solutions of inhomogeneous systems of algebraic equations, with applications to cyclic n-roots. *Journal of Symbolic Computation*, 12(3):329–336, 1991.

[4] W. Böge, Rüdiger Gebauer, and Heinz Kredel. Some examples for solving systems of algebraic equations by calculating Gröbner bases. *Journal of Symbolic Computation*, 2(1):83–98, 1986.

[5] Paul Breiding and Sascha Timme. HomotopyContinuation.jl: A package for homotopy continuation in Julia. In *Mathematical Software – ICMS 2018*, volume 10931 of *Lecture Notes in Computer Science*, pages 458–465. Springer, 2018.

[6] Bruno Buchberger. Bruno Buchberger's PhD thesis 1965: An algorithm for finding the basis elements of the residue class ring of a zero dimensional polynomial ideal. *Journal of Symbolic Computation*, 41(3–4):475–511, 2006.

[7] David A. Cox, John Little, and Donal O'Shea. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer, Cham, 4 edition, 2015.

[8] Wolfram Decker, Gert-Martin Greuel, Gerhard Pfister, and Hans Schönemann. Singular 4-4-1: A computer algebra system for polynomial computations. `https://www.singular.uni-kl.de`, 2025.

[9] Jean-Charles Faugère. A new efficient algorithm for computing Gröbner bases (F4). *Journal of Pure and Applied Algebra*, 139(1–3):61–88, 1999.

[10] Jean-Charles Faugère. A new efficient algorithm for computing Gröbner bases without reduction to zero (F5). In *Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation*, pages 75–83. ACM, 2002.

[11] Shuhong Gao, Frank Volny, IV, and Mingsheng Wang. A new framework for computing Gröbner bases. *Mathematics of Computation*, 85(297):449–465, 2016.

[12] Patrizia Gianni, Barry Trager, and Gail Zacharias. Gröbner bases and primary decomposition of polynomial ideals. *Journal of Symbolic Computation*, 6(2–3):149–167, 1988.

[13] Daniel R. Grayson and Michael E. Stillman. Macaulay2, a software system for research in algebraic geometry. `https://macaulay2.com`, 2026. Version 1.26.06.

[14] Hiroshi Kera, Yuki Ishihara, Yuta Kambe, Tristan Vaccon, and Kazuhiro Yokoyama. Learning to compute Gröbner bases. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[15] Martin Kreuzer and Lorenzo Robbiano. *Computational Commutative Algebra 1*. Springer, Berlin, 2000.

[16] Rusydi H. Makarim and Marc Stevens. M4GB: An efficient Gröbner-basis algorithm. In *Proceedings of the 2017 ACM International Symposium on Symbolic and Algebraic Computation*, pages 293–300. ACM, 2017.

[17] João Marques-Silva, Inês Lynce, and Sharad Malik. Conflict-driven clause learning SAT solvers. In *Handbook of Satisfiability*, volume 185 of *Frontiers in Artificial Intelligence and Applications*, pages 131–153. IOS Press, 2009.

[18] João P. Marques-Silva and Karem A. Sakallah. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521, 1999.

[19] Ernst W. Mayr and Albert R. Meyer. The complexity of the word problems for commutative semigroups and polynomial ideals. *Advances in Mathematics*, 46(3):305–329, 1982.

[20] Dylan Peifer, Michael Stillman, and Daniel Halpern-Leistner. Learning selection strategies in Buchberger's algorithm. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7575–7585. PMLR, 2020.

[21] Safi Rafie-Zinedine. Simplifying quantum gravity calculations. Master's thesis, Lund University, 2018. arXiv:1808.06086.

[22] Ben Ruijl. Symbolica: Modern computer algebra. `https://symbolica.io`, 2023. Version 1.5.

[23] Andrew J. Sommese, Jan Verschelde, and Charles W. Wampler. Numerical decomposition of the solution sets of polynomial systems into irreducible components. *SIAM Journal on Numerical Analysis*, 38(6):2022–2046, 2001.

[24] Andrew J. Sommese and Charles W. Wampler, II. *The Numerical Solution of Systems of Polynomials Arising in Engineering and Science*. World Scientific, Singapore, 2005.

[25] Bernd Sturmfels. *Solving Systems of Polynomial Equations*, volume 97 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, Providence, RI, 2002.

[26] Xiaojing Sun, Ioana Ilioaea, Priyank Kalla, and Florent Enescu. Finding unsatisfiable cores of a set of polynomials using the Gröbner basis algorithm. In *Principles and Practice of Constraint Programming*, volume 9892 of *Lecture Notes in Computer Science*, pages 859–875. Springer, 2016.

[27] Gian Paolo Vacca and Omar Zanusso. Asymptotic safety in einstein gravity and scalar-fermion matter. *Physical Review Letters*, 105(23):231601, 2010.

A Complete experimental results

The tables below give every run summarized in Table 2. Unless stated otherwise, entries are end-to-end wall times in seconds. P marks a partial run and TO an outer timeout. These are single measurements.

A.1 Factor branching

Table 4 reports all five branch-solver configurations. A partial entry found the easy component shown in the main text but left at least one cover unresolved.

System	M4GB	F5	GVW	Symbolica	Singular
Signed distance, $n = 8$	0.651	0.053	0.122	0.008	0.083
Signed distance, $n = 10$	0.046	10.927	45.797	0.025	0.084
Signed distance, $n = 12$	0.182	6.256	1.841	0.087	0.271
Overlapping $(4, 6, 2)$	0.017	0.008	0.011	0.015	0.038
Overlapping $(6, 8, 3)$	1.053	0.132	0.280	0.657	0.201
Overlapping $(8, 10, 3)$	TO	10.937	30.687	TO	2.931
Positive affine $(8, 6, 4)$	0.009	0.008	0.010	0.008	0.497
Positive affine $(10, 8, 5)$	0.021	0.020	0.028	0.020	1.870
Positive affine $(12, 10, 6)$	0.072	0.067	0.096	0.068	6.498
Gated Cyclic-5	0.010	0.010	0.011	0.009	0.022
Gated Cyclic-6	P 60.010	5.960	2.275	P 60.013	0.045
Gated Cyclic-7	P 60.875	P 60.504	P 60.955	P 60.024	P 60.103
Gated Katsura-6	0.366	0.483	1.338	2.540	0.049
Gated Katsura-7	4.487	15.220	42.113	P 60.012	0.283
Gated Katsura-8	54.055	P 60.325	P 60.047	P 60.012	2.657
Gated dense, $n = 5$	0.177	0.071	0.200	0.204	0.029
Gated dense, $n = 6$	4.716	3.194	7.475	21.498	0.546
Gated dense, $n = 7$	P 60.008	P 60.050	P 60.041	P 60.015	42.020
Square affine $(8, 4)$	15.117	TO	TO	3.096	0.470
Square affine $(10, 5)$	14.750	TO	47.561	1.174	2.548

Table 4: End-to-end wall times for every factor-branching configuration.

A.2 Variable zeroing

Table 5 gives the complete output of the fastest completed zeroing configuration used in Table 2. Every backend receives three seconds per cover and a 60-second outer budget, and UNSAT-core learning is disabled. The overlapping-random-multiple rows disable interreduction, as in the ordinary branching experiment. Here N is the number of recursion nodes, E the number of cover evaluations, O/S the numbers of per-cover timeouts and zeroing splits, and C/U the numbers of returned components and unresolved branches. The time entry is T/F.

System	Backend	T/F	N	E	O/S	C/U	Result
Signed distance, $n = 8$	Symbolica	0.008/0.001	1	8	0/0	8/0	complete
Signed distance, $n = 10$	Symbolica	0.025/0.002	1	8	0/0	8/0	complete
Signed distance, $n = 12$	Symbolica	0.091/0.003	1	32	0/0	32/0	complete
Overlapping $(4, 6, 2)$	F5	0.009/0.001	1	4	0/0	4/0	complete
Overlapping $(6, 8, 3)$	F5	0.132/0.021	1	16	0/0	16/0	complete
Overlapping $(8, 10, 3)$	Singular	2.892/0.152	1	28	0/0	28/0	complete
Positive affine $(8, 6, 4)$	Singular	0.181/0.012	1	4	0/0	4/0	complete
Positive affine $(10, 8, 5)$	Symbolica	0.060/0.007	1	242	0/0	242/0	complete
Positive affine $(12, 10, 6)$	Singular	9.600/0.078	1	511	0/0	511/0	complete
Gated Cyclic-5	Symbolica	0.011/0.001	1	2	0/0	2/0	complete
Gated Cyclic-6	Singular	0.043/0.009	1	2	0/0	2/0	complete
Gated Cyclic-7	Singular	24.118/0.009	15	17	8/7	1/1	partial
Gated Katsura-6	Singular	0.049/0.009	1	2	0/0	2/0	complete
Gated Katsura-7	Singular	0.284/0.009	1	2	0/0	2/0	complete
Gated Katsura-8	Singular	2.588/0.009	1	2	0/0	2/0	complete
Gated dense, $n = 5$	Singular	0.029/0.008	1	2	0/0	2/0	complete
Gated dense, $n = 6$	Singular	0.535/0.008	1	2	0/0	2/0	complete
Gated dense, $n = 7$	Singular	4.797/–	–	–	–	–	error
Square affine $(8, 4)$	Singular	0.445/0.012	1	31	0/0	31/0	complete
Square affine $(10, 5)$	Symbolica	1.326/0.015	1	304	0/0	304/0	complete

Table 5: Selected variable-zeroing result for every system. Times are end-to-end wall time and first-solution latency in seconds.

The dense $n = 7$ run failed while constructing the first recursive child: the expanded Singular program exceeded the operating system's command-line argument limit. This is an implementation error, not an UNSAT result or a timeout. We also tried M4GB, F5, GVW, and Symbolica on this system. These backends execute in-process; after a per-cover timeout their non-cancellable worker threads continued alongside the recursive children until the process was killed under memory pressure. Singular remains preferable for the hard rows because its external worker can be terminated. The Cyclic-7 run exhausted all useful splitting variables on one route and therefore returned one unresolved branch.

A.3 Direct whole-ideal Gröbner bases

System	M4GB	F5	GVW	Symbolica	Singular	Macaulay2
Signed distance, $n = 8$	0.646	0.014	0.010	0.008	0.333	0.570
Signed distance, $n = 10$	0.130	0.221	0.112	0.029	0.194	0.518
Signed distance, $n = 12$	3.266	20.044	8.685	0.472	0.512	1.293
Overlapping $(4, 6, 2)$	1.000	24.865	0.472	0.413	TO	TO
Overlapping $(6, 8, 3)$	TO	TO	TO	TO	TO	TO
Overlapping $(8, 10, 3)$	TO	TO	TO	TO	TO	TO
Positive affine $(8, 6, 4)$	TO	TO	TO	TO	TO	TO
Positive affine $(10, 8, 5)$	TO	TO	TO	TO	TO	TO
Positive affine $(12, 10, 6)$	TO	TO	TO	TO	TO	TO
Gated Cyclic-5	0.012	0.279	0.009	0.008	0.183	0.480
Gated Cyclic-6	TO	TO	1.738	TO	0.309	0.917
Gated Cyclic-7	TO	TO	TO	TO	TO	TO
Gated Katsura-6	0.436	44.961	0.226	5.277	14.430	TO
Gated Katsura-7	5.165	TO	3.176	TO	TO	TO
Gated Katsura-8	TO	TO	TO	TO	TO	TO
Gated dense, $n = 5$	0.171	8.884	0.179	0.200	0.529	TO
Gated dense, $n = 6$	4.736	TO	6.755	17.472	TO	TO
Gated dense, $n = 7$	TO	TO	TO	TO	TO	TO
Square affine $(8, 4)$	TO	TO	TO	TO	TO	TO
Square affine $(10, 5)$	TO	TO	TO	TO	TO	TO

Table 6: End-to-end wall times for every direct whole-ideal Gröbner-basis configuration.

A.4 Decomposition and numerical methods

System	facstd	primdecGTZ	Macaulay2	Homotopy
Signed distance, $n = 8$	0.190	0.320	1.020	32.410
Signed distance, $n = 10$	0.430	0.667	3.224	29.944
Signed distance, $n = 12$	4.800	3.056	25.223	36.880
Overlapping $(4, 6, 2)$	0.265	TO	TO	30.069
Overlapping $(6, 8, 3)$	TO	TO	TO	TO
Overlapping $(8, 10, 3)$	TO	TO	TO	TO
Positive affine $(8, 6, 4)$	0.368	TO	TO	50.934
Positive affine $(10, 8, 5)$	1.836	TO	TO	TO
Positive affine $(12, 10, 6)$	53.318	TO	TO	TO
Gated Cyclic-5	0.181	0.337	0.709	35.054
Gated Cyclic-6	2.729	TO	2.620	37.314
Gated Cyclic-7	TO	TO	TO	58.287
Gated Katsura-6	0.233	TO	TO	31.532
Gated Katsura-7	0.539	TO	TO	31.980
Gated Katsura-8	3.596	TO	TO	34.264
Gated dense, $n = 5$	0.198	TO	TO	29.289
Gated dense, $n = 6$	0.755	TO	TO	30.053
Gated dense, $n = 7$	48.820	TO	TO	32.261
Square affine $(8, 4)$	0.601	TO	TO	TO
Square affine $(10, 5)$	2.141	TO	TO	TO

Table 7: End-to-end wall times for every decomposition and numerical configuration.